82. Указатели и массивы

В языке C++ указатели и массивы тесно связаны между собой.

Оглавление:

1. Сходства между указателями и массивами
2. Различия между указателями и массивами
3. Передача массивов в функции
4. Передача по адресу
5. Массивы в структурах и классах

Сходства между указателями и массивами

Фиксированный массив определяется следующим образом:

```
int array[4] = { 5, 8, 6, 4 }; // определяем фиксированный массив из 4 целых чисел
```

Для нас это массив из 4 целых чисел, но для компилятора `array` является переменной типа `int[4]`. Мы знаем что `array[0] = 5`, `array[1] = 8`, `array[2] = 6` и `array[3] = 4`. Но какое значение имеет сам `array`?

Переменная `array` содержит адрес первого элемента массива, как если бы это был **указатель**! Например:

```
#include <iostream>

int main()
{
    int array[4] = { 5, 8, 6, 4 };

    // Выводим значение массива (переменной array)
    std::cout << "The array has address: " << array << '\n';

    // Выводим адрес элемента массива
    std::cout << "Element 0 has address: " << &array[0] << '\n';

    return 0;
}
```

Результат на моем компьютере:

```
The array has address: 0048F968
Element 0 has address: 004BF968
```

Обратите внимание, адрес, хранящийся в переменной `array`, является адресом первого элемента массива.

Распространенная ошибка думать, что переменная `array` и указатель на `array` являются одним и тем же объектом. Это не так. Хотя оба указывают на первый элемент массива, информация о типе данных у них разная. В вышеприведенном примере типом переменной `array` является `int[4]`, тогда как типом указателя на массив является `int *`.

Путаница вызвана тем, что во многих случаях, при вычислении, фиксированный массив **распадается** (**неявно преобразовывается**) в указатель на первый элемент массива. Доступ к элементам по-прежнему осуществляется через указатель, но информация, полученная из типа массива (например, его размер), не может быть доступна из типа указателя.

Однако и это не является столь весомым аргументом, чтобы рассматривать фиксированные массивы и указатели как разные значения. Например, мы можем разыменовать массив, чтобы получить значение первого элемента:

```
int array[4] = { 5, 8, 6, 4 };

// Разыменовываем массива (переменной array) приведет к возврату первого элемента массива (элемента под номером 0)
std::cout << *array; // выведется 5!

char name[] = "John"; // строка C-style (также массив)
std::cout << *name; // выведется 'J'
```

Обратите внимание, мы не разыменовываем фактический массив. Массив (типа `int[4]`) неявно конвертируется в указатель (типа `int *`), и мы разыменовываем указатель, который указывает на значение первого элемента массива.

Также мы можем создать указатель и присвоить ему `array`:

```
#include <iostream>

int main()
{
    int array[4] = { 5, 8, 6, 4 };
    std::cout << *array; // выведется 5

    int *ptr = array;
    std::cout << *ptr; // выведется  5

    return 0;
}
```

Это работает из-за того, что переменная `array` распадается в указатель типа `int *`, а тип нашего указателя такой же (т.е. `int *`).

Различия между указателями и массивами

Однако есть случаи, когда разница между фиксированными массивами и указателями имеет значение. Основное различие возникает при использовании **оператора sizeof**. При использовании в фиксированном массиве, оператор sizeof возвращает размер всего массива (`длина_массива * размер_элемента`). При использовании с указателем, оператор sizeof возвращает размер адреса памяти (в байтах). Например:

```
#include <iostream>

int main()
{
    int array[4] = { 5, 8, 6, 4 };

    std::cout << sizeof(array) << '\n'; // выведется sizeof(int) * длина array

        int *ptr = array;
        std::cout << sizeof(ptr) << '\n'; // выведется размер указателя

    return 0;
}
```

Результат выполнения программы:

```
16
4
```

Фиксированный массив знает свою длину, а указатель на массив — нет.

Второе различие возникает при использовании оператора адреса `&`. Используя адрес указателя, мы получаем адрес памяти переменной указателя. Используя адрес массива, возвращается указатель на целый массив. Этот указатель также указывает на первый элемент массива, но информация о типе отличается. Вряд ли вам когда-нибудь понадобится это использовать.

Передача массивов в функции

Мы говорили, что, из-за того, что копирование больших массивов при передаче в функцию является очень затратной операцией, C++ не копирует массив. При передаче массива в качестве аргумента в функцию, массив распадается в указатель на массив и этот указатель передается в функцию:

```
#include <iostream>

void printSize(int *array)
{
    // Здесь массив рассматривается как указатель
    std::cout << sizeof(array) << '\n'; // выведется размер указателя, а не длина массива!
}

int main()
{
    int array[] = { 1, 2, 3, 4, 4, 9, 15, 25 };
    std::cout << sizeof(array) << '\n'; // выведется sizeof(int) * длина массива

    printSize(array); // здесь аргумент array распадается на указатель

     return 0;
}
```

Результат выполнения программы:

```
32
4
```

Обратите внимание, результат будет таким же, даже если параметром будет фиксированный массив:

```
#include <iostream>

// C++ неявно конвертирует параметр array[] в *array
void printSize(int array[])
{
    // Здесь массив рассматривается как указатель, а не как фиксированный массив
    std::cout << sizeof(array) << '\n'; // выведется размер указателя, а не размер массива!
}

int main()
{
    int array[] = { 1, 2, 3, 4, 4, 9, 15, 25 };
    std::cout << sizeof(array) << '\n'; // выведется sizeof(int) * длина массива array

    printSize(array); // здесь аргумент array распадается на указатель

     return 0;
}
```

Результат выполнения программы:

```
32
4
```

В примере, приведенном выше, C++ неявно конвертирует параметр из синтаксиса массива (`[]`) в синтаксис указателя (`*`). Это означает, что следующие два объявления функции идентичны:

```
void printSize(int array[]);
void printSize(int *array);
```

Некоторые программисты предпочитают использовать синтаксис `[]`, так как он позволяет понять, что функция ожидает массив, а не указатель на значение. Однако, в большинстве случаев, поскольку указатель не знает, насколько велик массив, вам придется передавать размер массива в качестве отдельного параметра (строки являются исключением, так как они нуль-терминированные).

Рекомендуется использовать синтаксис указателя, поскольку он позволяет понять, что параметр будет обработан как указатель, а не как фиксированный массив, и определенные операции, такие как в случае с оператором sizeof, будут выполняться с параметром-указателем (а не с параметром-массивом).

> ***Совет:*** Используйте синтаксис указателя (`*`) вместо синтаксиса массива (`[]`) при передаче массивов в качестве параметров в функции.

Передача по адресу

Тот факт, что массивы распадаются на указатели при передаче в функции, объясняет основную причину, по которой изменение массива в функции приведет к изменению фактического массива. Рассмотрим следующий пример:

```
#include <iostream>

// Параметр ptr получает копию адреса массива
void changeArray(int *ptr)
{
    *ptr = 5; // поэтому изменение элемента массива приведет к изменению фактического массива
}

int main()
{
    int array[] = { 1, 2, 3, 4, 4, 9, 15, 25 };
    std::cout << "Element 0 has value: " << array[0] << '\n';

    changeArray(array);

    std::cout << "Element 0 has value: " << array[0] << '\n';

     return 0;
}
```

Результат выполнения программы:

```
Element 0 has value: 1
Element 0 has value: 5
```

При вызове функции changeArray(), массив распадается на указатель, а значение этого указателя (адрес памяти первого элемента массива) копируется в параметр `ptr` функции changeArray(). Хотя значение `ptr` в функции является копией адреса массива, `ptr` всё равно указывает на фактический массив (а не на копию!). Следовательно, при разыменовании `ptr`, разыменовывается и фактический массив!

Этот феномен работает так же и с указателями на значения не из массива. Более детально эту тему мы рассмотрим на соответствующем уроке.

Массивы в структурах и классах

Стоит упомянуть, что массивы, которые являются частью **структур** или классов, не распадаются, когда вся структура или класс передается в функцию.

На следующем уроке мы рассмотрим адресную арифметику и поговорим о том, как на самом деле работает индексация массива.